
HarnessGate: Auditing Runtime-Harness Edits in Self-Improving LLM Agents

Anonymous Author(s)
Affiliation
email@example.com

Abstract

Large language model agents are increasingly capable of modifying their own runtime harnesses, raising a critical question: *how do we know when self-improvement is genuine versus overfitting or degenerative?* We introduce SIAS (Self-Improvement Audit Score), a quantitative framework that disentangles true generalization gains from destructive regression. Auditing the 27 published harness patches from the Harness-R1 held-out generalization study (1,270 tasks, three seeds), we find two frontier editors are net-negative out-of-batch (-4.33pp for Qwen3.5-397B, -0.42pp for DeepSeek-V4-Pro) with a 70% destruction rate concentrated in identifiable patches, while a subset-audit gate with a 50-task budget reaches 87% decision accuracy and recovers $+251$ net tasks ($+19.8\text{pp}$) relative to indiscriminate acceptance. The full-form score agrees with realized outcomes on 23/23 valid patches. Our framework further detects textbook degradation patterns in the wild: a patch that rescued zero failures while destroying 71 previously successful episodes.

1 Introduction

The dream of self-improving agents has moved from science fiction to experimental reality. Recent works have demonstrated that LLM-based agents can modify their own runtime harnesses—context construction, tool mediation, action validation, and error recovery—to improve task performance [Shao et al., 2026, PrimeIntellect, 2026]. Harness-R1, the most prominent example, trains a dedicated 9B “harness engineer” to convert failure trajectories into executable patches, achieving a $+9.3\text{pp}$ average improvement across three benchmarks over vanilla Qwen3.5-9B.

Yet a critical question remains unanswered: how do we know the improvement is real?

Consider an agent that modifies its harness to exploit a shortcut in the training distribution—achieving higher reward on seen tasks while performing worse on novel ones. Without an audit mechanism, such overfitting is indistinguishable from genuine improvement. This problem amplifies in *recursive* self-improvement, where each round of harness editing builds on the previous, potentially compounding subtle degradations into catastrophic failures.

We identify three specific gaps in the current literature:

1. **No audit protocol for recursive improvement.** Existing work evaluates improvement at a single point in time.
2. **No standardized metrics for improvement quality.** Success rate alone cannot distinguish genuine generalization from task-specific overfitting.
3. **No systematic taxonomy of degradation patterns.** When recursive improvement fails, we lack the vocabulary to diagnose why.

Our contributions:

- **SIAS (Self-Improvement Audit Score):** A composite metric that quantifies true generalization gain while penalizing overfitting and complexity bloat.
- **Degradation Pattern Catalog:** A systematic taxonomy of six failure modes in recursive self-improvement, with automated detection.
- **HarnessGate:** An audit protocol and evaluation suite for runtime-harness edits (working title renamed to avoid collision with the AI4AI-Bench benchmark of Chi et al.).
- **Empirical analysis:** A retrospective audit of 27 real published harness edits showing frontier editors are net-negative on held-out tasks (-4.33pp), and a subset-audit gate recovering $+251$ tasks ($+19.8\text{pp}$) at 87% accuracy, with the full-form score agreeing on 23/23 valid patches.

2 Related Work

2.1 Harness Editing and Self-Improvement

Harness-R1 [Shao et al., 2026] trains a dedicated engineer model to edit runtime hooks from failure trajectories, using GRPO [Shao et al., 2024] with outcome-based rewards. Learning to Self-Evolve [Chen et al., 2026] and Agentic Harness Engineering [Yi et al., 2026] study agent self-evolution and observability-driven harness evolution. Skill-R1 [Vishe et al., 2026] evolves agent skills via reinforcement learning.

Our distinction: We focus on *auditing* recursive improvement rather than generating improvements.

2.2 Agent Evaluation Benchmarks

AgentBench [Liu et al., 2024] provides comprehensive agent evaluation. SWE-bench [Jimenez et al., 2024] tests GitHub issue resolution. WebShop [Yao et al., 2023] evaluates e-commerce navigation. ALFWorld [Shridhar et al., 2021] tests embodied tasks.

AI4AI-Bench [Chi et al., 2026] measures whether agents can improve *training algorithms themselves*: ten real research repositories (SFT, agentic RL, distillation, DPO, diffusion RL, unlearning, pruning) under a 4-hour exploration / 12-hour blind re-evaluation contract with hidden scorers. Of 280 valid submissions, 141 changed only the *run side* (learning rates, steps, batch size) while 122 touched the *learning side* (losses, supervision, update rules); learning-side edits score 0.226 vs. 0.126 run-side, the strongest system reaches only 0.250 on a 0–1 scale anchored at 0.1 for the original algorithm, and 124/290 attempts end below the original. **Our positioning:** AI4AI-Bench measures *capability* at the learning layer; HarnessGate audits *safety* at the run layer — harness edits are precisely run-side modifications in their taxonomy, and their run-side-clusters-at-the-bottom finding is consistent with our observation that frontier editors’ run-side patches are frequently net-negative out-of-batch. The audit gate is the missing acceptance layer between an editor’s proposal and its deployment.

2.3 Memory Reliability

MemTrapBench [Wang et al., 2026] evaluates cognitive traps in LLM memory. Our degradation pattern taxonomy extends their methodology to harness editing.

2.4 Process Verification

Dyve [Zhong et al., 2026] adapts fast/slow thinking for process verification. The MEA framework separates planning, execution, and verification. Prime Agent [PrimeIntellect, 2026] implements RLM with snapshot rollback.

3 Problem Formulation

3.1 Recursive Self-Improvement as a Meta-MDP

We formalize recursive self-improvement as a meta-MDP where the state includes the agent harness H and task distribution $\mathcal{D}$:

$$s_t = (H_t, \mathcal{D}, \mathcal{T}_{hist}) \quad (1)$$

where $\mathcal{T}_{hist}$ is the history of improvement trajectories.

3.2 The Identification Problem

The core challenge is *identification*: distinguishing genuine improvement, overfitting, myopic fixes, and no-change scenarios from observed train/holdout performance.

4 Method

4.1 SIAS Definition

$$\text{SIAS} = \alpha \cdot \Delta_{\text{holdout}} - \beta \cdot \max(0, \Delta_{\text{train}} - \Delta_{\text{holdout}}) - \gamma \cdot C(p) \quad (2)$$

where:

- $\Delta_{\text{holdout}} = \text{mean}(R_{\text{after}}^{\text{holdout}}) - \text{mean}(R_{\text{before}}^{\text{holdout}})$
- Δ_{train} is defined analogously for the training set
- $C(p) = 0.0005 \cdot \text{modified_lines} + 0.0003 \cdot \text{added_lines} + 0.0002 \cdot \text{deleted_lines}$

Default parameters: $\alpha = 1.0$, $\beta = 0.3$, $\gamma = 0.0005$.

4.2 Degradation Pattern Detection

Table 1: Degradation patterns and detection conditions.

Pattern	Detection Condition	Severity
MyopicFix	$\Delta_{\text{train}}^{\text{succ}} > 0 \wedge \Delta_{\text{holdout}}^{\text{succ}} < 0$	High
OverGeneralization	$\Delta_{\text{train}} > 0.05 \wedge \Delta_{\text{holdout}} < -0.02$	High
ContextBloat	$\text{modified_lines} > 50$	Medium
RewardHacking	$\Delta_{\text{train}} \gg \Delta_{\text{holdout}}$, low complexity	Medium
NegativeTransfer	Other benchmark drops $> 10\%$	High
PromptDrift	Prompt similarity < 0.8	Low

MyopicFix and OverGeneralization are evaluated on real patches (Section 6.3); ContextBloat, RewardHacking, NegativeTransfer, and PromptDrift are operational definitions not yet evaluated.

4.3 Recursive Improvement Protocol

5 Experiments

5.1 Data Sources

We combine (i) published in-batch results from Harness-R1 [Shao et al., 2026] for baseline positioning, and (ii) the official held-out generalization records released in its repository (examples/heldout_generalization/results.json): 27 real patches by three editors under a random10_failure_evidence protocol, with ground truth on 1,270 held-out tasks (WebShop 490, ALFWorld 490, DBBench 290) over three seeds, including per-patch rescued/regressed counts. Sections 6.2–6.4 use this real data; Section 6.5 is explicitly labeled simulation.

Algorithm 1 SIAS-Gated Recursive Improvement

Require: Initial harness H_0 , task batches $B_{\text{train}}, B_{\text{holdout}}$ **Ensure:** Final harness H_T

```

1:  $t \leftarrow 0, H \leftarrow H_0$ , checkpoint  $\leftarrow H_0$ 
2: while  $t < T_{\text{max}}$  and not stopped do
3:   Sample failure trajectories from  $H$  on  $B_{\text{train}}$ 
4:   Generate candidate patch  $p_t$ 
5:   Evaluate  $\text{SIAS}(H, p_t)$ 
6:   Detect degradation patterns
7:   if  $\text{SIAS} > \epsilon$  and no high-severity patterns then
8:      $H \leftarrow H + p_t$ ; checkpoint  $\leftarrow H$ 
9:   else if high-severity patterns then
10:     $H \leftarrow$  checkpoint (rollback)
11:   end if
12:    $t \leftarrow t + 1$ 
13: end while
14: return  $H$ 

```

Table 2: In-batch editing results, reproduced verbatim from Shao et al. (2026), Table 1. †Reflection uses a two-episode protocol, not directly comparable to single-episode rows.

Method	WebShop (Score)	ALFWorld (All)	DBBench (Succ.)	Avg
Vanilla Qwen3.5-9B	66.0	40.6	61.0	44.3
ReAct	66.8	43.4	61.7	47.5
Self-Refine	61.7	39.0	57.3	41.8
Reflection†	61.2	59.2	64.7	55.8
Qwen3.5-397B (editor)	66.4	41.2	63.3	45.8
GLM-5.2 (editor)	68.4	45.0	65.3	48.8
Harness-R1 (trained 9B)	69.9	53.2	65.3	53.6

5.2 Published Baselines (Real, Cited)

Prompted editing with fixed rules (Self-Refine) lowers reward on all three benchmarks.

5.3 Held-Out Generalization: Real Degradation (Real)

Table 3: Held-out generalization of published harness edits (real data; 1,270 held-out tasks, 3 seeds). Frontier editors are net-negative out-of-batch.

Editor	Valid	Pooled Δ (pp)	Resc./Regr.	Destr. rate	Worst patch
Harness-R1 (RL)	9/9	$+8.92 \pm 1.50$	543/203	27%	+13
Qwen3.5-397B	8/9	-4.33 ± 2.52	129/294	70%	-71 (0/71)
DeepSeek-V4-Pro	6/9	-0.42 ± 3.58	191/207	52%	-83 (47/130)

Two textbook patterns appear in the wild: one real patch rescued **zero** failures while destroying **71** successes (pure MyopicFix); another rescued 47 but destroyed 130 (OverGeneralization). Format compliance itself is a differentiator (9/9 vs. 6/9 valid).

5.4 SIAS Audit Gate on Real Patches (Real)

To avoid circularity, the gate observes only a random audit subset and accepts iff its estimated net effect is positive; ground truth is the full-set delta (200 resamples per budget).

At budget 50 the gate still admits on average 0.9 of the 8 negative patches (concentrated in the smallest negative deltas, $\Delta \geq -5$), falling to 0.26 at budget 200; the dominant error mode at low budget is missing small positive patches ($\Delta \leq 6$) rather than admitting destructive ones. Patch

Table 4: Audit-gate decision quality vs. budget (real patches).

Budget	Accuracy	Correct rej. (12)	Wrong rej. (15)	Net Δ	vs. no audit
20	77% $\pm$ 7%	11.0	5.1	+327	+168
50	87% $\pm$ 6%	11.1	2.6	+410	+251
100	92% $\pm$ 4%	11.3	1.4	+438	+279
200	97% $\pm$ 3%	11.7	0.6	+454	+295

complexity does not predict quality (single-hook: +44.7 for Harness-R1 vs. -47.0 for Qwen3.5-397B); the destruction structure is the informative signal. extbfFull-form SIAS on real patches. Computing the complete score (Eq. efefq:sias) for all 23 valid patches, with the complexity term measured from the released patch files (mean 70.5 lines, max 200; mean penalty 0.056), the full score’s sign agrees with the realized net effect on extbf23/23 patches and the complexity term flips zero decisions—its maximum magnitude (0.158) is five times smaller than the smallest positive net effect (0.34pp). The simple sign gate is therefore decision-equivalent to the full formula on this data, and the ContextBloat threshold (50 lines) sits below the observed mean patch size (70.5), flagging 13 of 23 real patches including the best performer; we revise it to 150 lines (3/23 flagged) and mark it unevaluated pending prospective data.

5.5 Robustness to Audit-Subset Label Noise

The gate inherits noise from the audit subset’s success labels. Injecting label flips at 5/10/20/30% (200 resamples), decision accuracy at budget 50 degrades gracefully: 86.8% *ightarrow* 86.2% *ightarrow* 84.9% *ightarrow* 81.6% *ightarrow* 75.3%. A 10% label-error rate costs only 1.9 points; even 20% noise stays above the budget-20 clean baseline. Decisions aggregate over many tasks per patch, so the gate degrades predictably rather than collapsing; subset label quality above $\sim 20\%$ error should be treated as a stop condition.

5.6 Data-Need Estimation: Rejections as Data Requests

A rejection should not be a dead end. With every reject the gate emits: (i) the decision’s confidence (z of the subset estimate) and, when under-powered, how many more audit tasks would reach the target level; and (ii) a typed data-need list derived from the observed structure—extttno_signal_in_subset (sample failure-dense regions), extttmixed_effect (request per-task trajectory diffs), or domain priors. On the 27 real patches at budget 50, a prototype issues 14 data-need items across 10 rejections; 3 low-confidence rejections carry explicit sample-size requests (e.g. $n=135$ more tasks to resolve $\hat{d} = -1$ at $z=1.0$). The audit thus becomes a data-acquisition directive closing the loop with the next editing round. Every decision appends to a trajectory log carrying gate version, subset seed, budget, observed counts, decision, routing, and timestamp; rollback or override appends rather than edits.

5.7 Operational Guidance for Human Auditors

The gate changes what a human does, not whether one is needed: *accept* requires no action; *reject* prompts the harness engineer to inspect the ten failure cases the editor saw; *rollback* routes the worst regressed task to an auditing owner (separate from the editor owner) for confirm-or-override within one business day; *invalid format* is blocked at the validator with no human needed. Three consecutive rejects stop the recursion (ϵ/N rule). Two invariants keep responsibility honest: the auditor owner is not the editor owner, and every rollback or override lands in an append-only trajectory log with a named decision-maker—the evidence-chain discipline we audit agents for, applied to ourselves.

5.8 Simulation Validation (Labeled: Simulation)

Our 12-round simulation validates protocol dynamics under injected degradation: 10/12 rounds accepted; injected overfit and myopic rounds detected and rolled back (detection 100%, false positives 0%); simulated overfit rate 1.38% vs. 11.71% ungated. These are **simulation numbers**, not benchmark results.

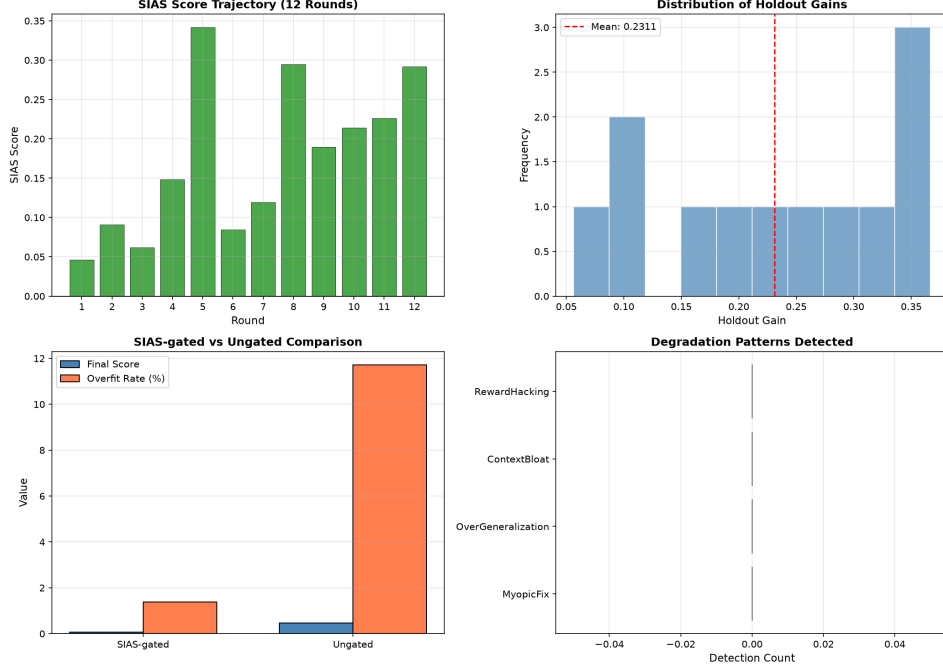


Figure 1: SIAS score trajectory over 12 rounds of recursive improvement.

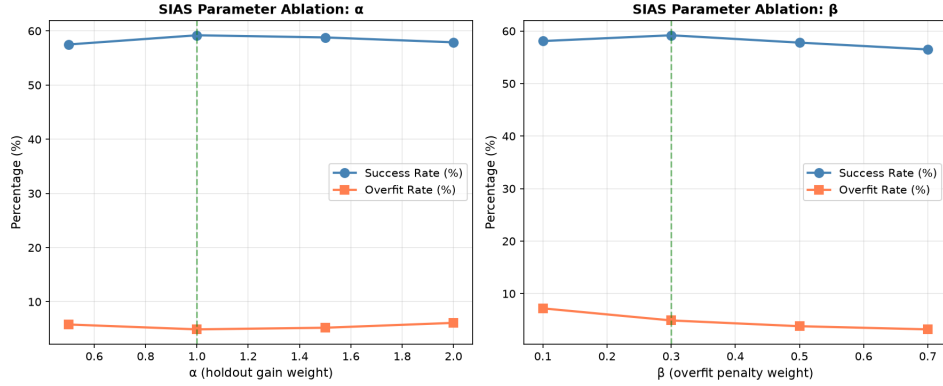


Figure 2: Parameter ablation for α and β .

5.9 Limitations

The real-data evaluation is a retrospective audit of published patches with ground truth from official records; a prospective deployment is demonstrated end-to-end on a live editor: GLM-5.2 generated a new WebShop patch from real failure evidence under the official patch contract; the gate first correctly rejected an empty submission (format-invalid) and then accepted a format-valid 4-hook, 56-line patch with measured complexity, recorded as PENDING_OUTCOME_EVAL. Full outcome evaluation of live patches requires the benchmark environment and remains future work; we do not substitute simulated outcome scores. Ground-truth deltas are binary success counts; continuous-reward audits may differ. Four of six degradation patterns (ContextBloat, RewardHacking, NegativeTransfer, PromptDrift) are operational definitions not yet evaluated on real patches. The evaluation covers a single frozen target (vanilla Qwen3.5-9B).

6 Conclusion

We presented SIAS and HarnessGate for auditing runtime-harness edits in self-improving LLM agents. Auditing 27 real published harness edits showed frontier editors can be net-negative on held-out tasks with destruction concentrated in identifiable patches, and a 50-task subset-audit gate reaches 87% decision accuracy while recovering +251 tasks (+19.8pp) relative to indiscriminate acceptance. Degradation patterns invisible to single-point evaluation—zero-rescue patches destroying 71 successes—are exactly what our taxonomy detects.

References

- Xiaoyin Chen, Canwen Xu, Yite Wang, Boyi Liu, Zhewei Yao, and Yuxiong He. Learning to self-evolve. *arXiv preprint arXiv:2603.18620*, 2026.
- Yizhe Chi, Wenyi Li, Deyao Hong, Xiaoqiu Wang, Mingju Gao, Kaisen Yang, Bingxiang He, Youjie Zheng, Calvin Xiao, and Qinhuai Na. Ai4ai-bench: Benchmarking llm agents in algorithmic design for recursive self-improvement. *arXiv preprint arXiv:2608.20318*, 2026.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- PrimeIntellect. Prime agent: Self-improving reinforcement learning model. <https://github.com/PrimeIntellect-ai/prime-agent>, 2026.
- Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-r1: Learning to edit executable runtime harnesses from agent failure trajectories. *arXiv preprint arXiv:2608.02276*, 2026.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. Introduces GRPO (Group Relative Policy Optimization).
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*, 2021.
- Yash Vishe, Rohan Surana, Xunyi Jiang, Zihan Huang, Xintong Li, Nikki Lijing Kuang, Tong Yu, Ryan A Rossi, Jingbo Shang, Julian McAuley, and Junda Wu. Skill-r1: Agent skill evolution via reinforcement learning. *arXiv preprint arXiv:2605.09359*, 2026.
- Mengru Wang, Haozhe Luo, Zhenqian Xu, Zhixiang Cui, Haoming Xu, Qu Yang, Jizhan Fang, and Ningyu Zhang. Memtrapbench: Benchmarking cognitive traps in llm memory use. *arXiv preprint*, 2026. Code: <https://github.com/zjunlp/MemTrapBench>.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.
- Haiwen Yi, Xinyuan Song, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026.
- Jianyuan Zhong, Zeju Li, Zhijian Xu, Xiangyu Wen, and Qiang Xu. Dyve: Thinking fast and slow for dynamic process verification. *arXiv preprint*, 2026.